Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	29 October 2026
Team ID	
Project Name	Automated Network Request Management
Maximum Marks	4 Marks

Technical Architecture:

Automated Network Request Management is a **ServiceNow scoped low-code application** built in ServiceNow App Engine Studio on a hosted Personal Developer Instance (PDI). It is not a traditional standalone full-stack application: every layer (user interface, workflow, data storage, approval, email and security) uses native ServiceNow platform capabilities. The architectural diagram (page 2) and the information in Table-1 (page 3) and Table-2 (page 4) describe the implementation as it currently exists. Components that are not part of the implementation are shown greyed out and labelled as not implemented. **Example scenario:** network service request handling (new / existing connection) with approval, task tracking and email notifications.

Architecture Guidelines Checklist

Guideline	How it is addressed in this project
Include all the processes (application logic / technology block)	Three logic blocks: Network Request flow (request processing), approval handling with If/Else update, and Network Task Completion Email flow (Table-1, rows 2-4).
Provide infrastructural demarcation (local / cloud)	Entire solution runs in the cloud inside a ServiceNow-hosted PDI (dashed green boundary in the diagram). No local server is used.
Indicate external interfaces (third-party APIs etc.)	None. No external API or third-party integration is configured.
Indicate data storage components / services	Two custom tables (Database Tables, Network Task Table) and the platform approval table (sysapproval_approver), all in the ServiceNow platform-managed database.
Indicate interface to machine learning models	Not applicable. No machine learning or AI model is used.

Data Flow (numbers match the diagram)

1	The requester opens the Service Portal (/sp) and selects the Network Request catalog item; the Catalog UI Policy shows Existing ID only for Existing connections.
2	The requester submits the form; the catalog request triggers the Network Request flow.
3	The flow gets the catalog variables and creates a Database Tables record (Approval Status = Requested; Database Number = requested item number).
4	The flow sends the request-created email (subject: Request has been Created; two business working days).
5	The flow asks for approval; an approval record is created and shown through the related list on the Database Tables record.
6	The approver's decision returns to the flow: Approved updates the record (ITIL User, Approved); the Else branch is configured to set Abraham Lincoln and Rejected.
7	**Manual step:** a Network Task Table record is created by a user for the request; the flow does not create it automatically.
8	When the task's Work Status changes to Completed, the Network Task Completion Email flow runs.
9	The flow sends the resolution email (subject: Request has been Resolved) to the Requested For user; it appears in the Outbox (state Ready).

Architecture Diagram: Automated Network Request Management on ServiceNow

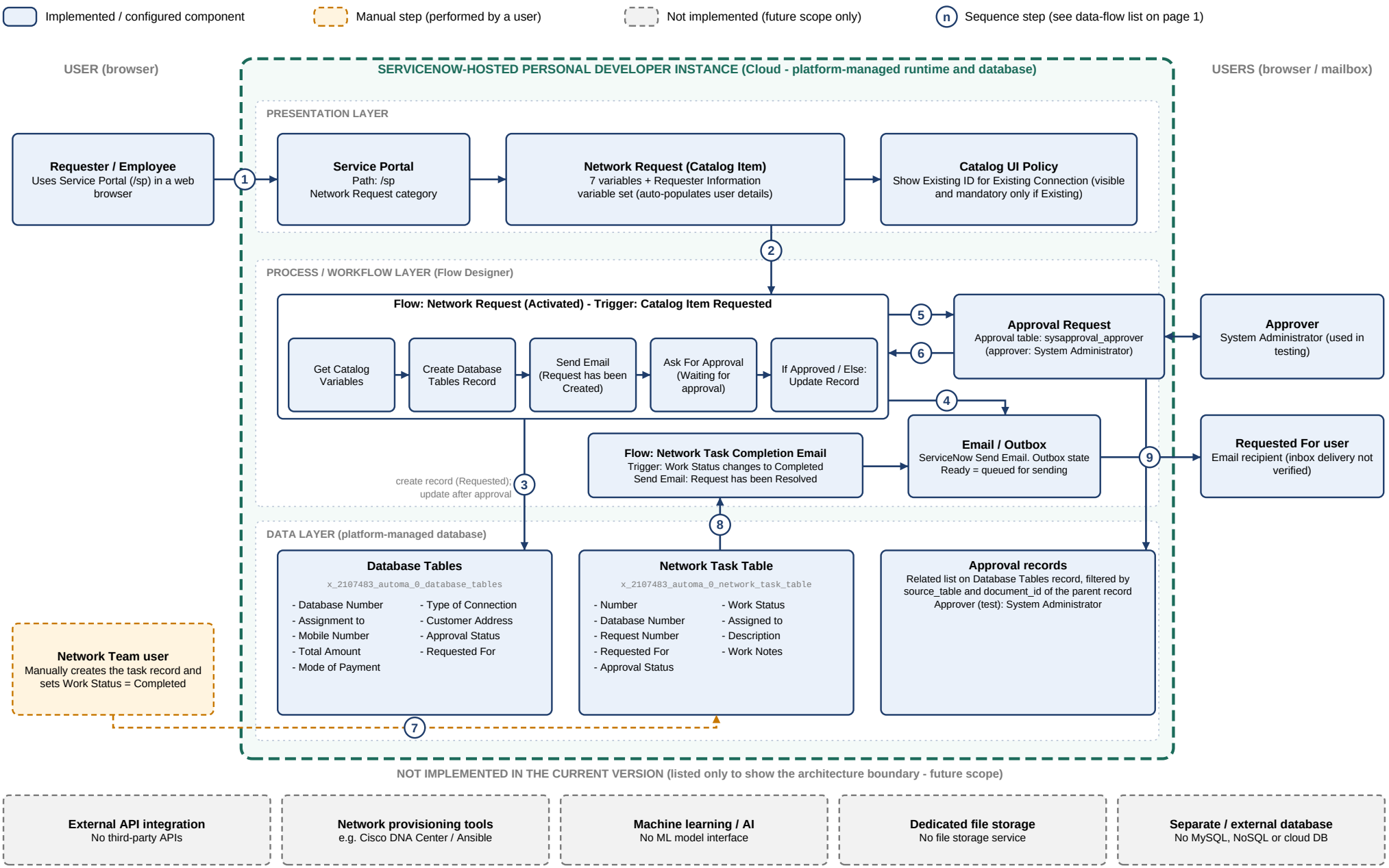

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	**User Interface**	Requester-facing interface. Users open the Service Portal (/sp), go to the Network Request catalog item and fill in Requested For, Mobile Number, Type of Connection, Existing ID, Total Amount, Mode of Payment and Address. A Requester Information variable set (Opened on behalf of, Email, User Name, Phone Number) fills user details; the Catalog UI Policy 'Show Existing ID for Existing Connection' shows and requires Existing ID only for Existing connections.	**Implemented:** ServiceNow Service Portal, Service Catalog, Catalog Item, Catalog Variables, Variable Set, Catalog UI Policy. No custom HTML / CSS / JavaScript front-end framework (such as Angular or React) is used.
2.	**Application Logic-1**	Request processing. The Network Request flow starts when the catalog item is requested. It gets the six catalog variables, creates a record in the Database Tables table (Approval Status = Requested) and sends the 'Request has been Created' email with the request number and a two business working day resolution message.	**Implemented:** ServiceNow Flow Designer (Get Catalog Variables, Create Record, Send Email actions).
3.	**Application Logic-2**	Approval handling. The flow asks for approval (reason: Waiting for approval; approver in testing: System Administrator). Approved: Assignment to = ITIL User, Approval Status = Approved. Else branch: Assignment to = Abraham Lincoln, Approval Status = Rejected (configured; not freshly tested). Approval records are shown on the Database Tables record through a related list filtered by source_table and document_id.	**Implemented:** Flow Designer (Ask For Approval, If/Else, Update Record); ServiceNow approval table sysapproval_approver; related-list filter script.
4.	**Application Logic-3**	Task completion notification. The Network Task Completion Email flow runs when a Network Task Table record's Work Status changes to Completed (for each unique change) and emails the Requested For user with subject 'Request has been Resolved'. The task record itself is created manually; it is not generated by the Network Request flow.	**Implemented:** Flow Designer (Record Updated trigger, Send Email action); ServiceNow Outbox. A separate Email Notification 'Network Task Resolved' is configured, but the demonstrated email came from this flow.
5.	**Database**	Two custom scoped tables. **Database Tables** (x_2107483_automa_0_database_tables) stores request data and Approval Status (9 fields). **Network Task Table** (x_2107483_automa_0_network_task_table) tracks work with a separate Work Status (9 fields). Reference fields link records to users, the catalog request and the Database Tables record.	**Implemented:** ServiceNow platform-managed database with custom tables created through App Engine Studio. No separately installed MySQL or NoSQL database.
6.	**Cloud Database**	No separate cloud database service is configured. All data is stored in the database managed by the ServiceNow-hosted instance.	ServiceNow-hosted platform database. **Not implemented:** IBM DB2, IBM Cloudant or any external cloud database.
7.	**File Storage**	The application does not store files or attachments as part of its design.	**Not implemented:** dedicated file storage.
8.	**External API-1**	No external API is used. All processing stays inside the ServiceNow instance.	**Not implemented.** (Possible future scope: Integration Hub.)
9.	**External API-2**	No second external API is used.	**Not implemented.** (Possible future scope: network provisioning tools.)
10.	**Machine Learning Model**	No machine learning model or AI inference is used in the application.	**Not implemented.**
11.	**Infrastructure (Server / Cloud)**	The scoped application is deployed and run in a ServiceNow-hosted Personal Developer Instance accessed through a web browser. **Local Server Configuration:** None (no local server). **Cloud Server Configuration:** ServiceNow-managed hosted instance; no custom application server. The PDI is a development prototype, not a production deployment.	**Implemented:** ServiceNow-hosted PDI (cloud, platform-managed). **Not implemented:** Docker, Kubernetes, Cloud Foundry, other cloud providers.

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	**Open-Source Frameworks**	No open-source framework is added by the project. The application is built entirely with ServiceNow's native low-code capabilities.	None. ServiceNow App Engine Studio (proprietary platform).
2.	**Security Implementations**	Security uses the scoped application boundary, application roles and Access Control Lists (ACLs). Generated/default ACLs exist for Create, Read, Write and Delete; those observed were active and use the Allow If condition type. Authentication is provided by the ServiceNow platform. This is a development-level configuration; ACLs should be reviewed for least-privilege access before any production use.	**Implemented:** application scope, application roles, ACLs, platform authentication. **Not implemented:** custom encryption, SHA-256, firewall, custom identity provider, penetration testing, security certification.
3.	**Scalable Architecture**	The solution is organised into presentation (Service Portal / catalog), process (Flow Designer flows) and data (custom tables) layers running on the ServiceNow platform's own application and database tiers. New catalog items, flows or tables can be added to the scoped app. Scalability of this prototype has not been load tested and is bounded by the instance and platform limits.	**Implemented:** platform-managed multi-tier ServiceNow architecture; modular Flow Designer flows. **Not implemented:** micro-services.
4.	**Availability**	Availability is provided only by the ServiceNow-hosted instance. The project does not configure load balancers or distributed servers, and a Personal Developer Instance is for development, so no production availability or SLA is claimed.	ServiceNow-hosted PDI. **Not implemented:** load balancing, high availability, failover.
5.	**Performance**	Design considerations: a short seven-variable request form; server-side processing by Flow Designer; and a narrow trigger on the completion flow (Work Status changes to Completed, run for each unique change) to avoid unnecessary executions. No requests-per-second figures or benchmarks are available. Earlier runs showed missing catalog-variable warnings; a later test record held the correct values, so wider regression and performance testing remain future work.	Flow Designer conditional trigger; ServiceNow runtime. **Not implemented:** caching, CDN, performance or load testing.

Assumptions and Limitations

- Request Number format (for example RITM0010009) and task numbers (for example NTT0000109) are examples from test runs, not fixed values.
- Approval Status (request-level: Requested, Approved, Rejected) and Work Status (task-level progress) are separate concepts.
- An Outbox state of Ready shows that ServiceNow generated and queued the email; it does not prove inbox delivery.
- Mobile Number is not checked by a custom ten-digit validation script; Existing ID is conditionally mandatory through the Catalog UI Policy action.
- Exact scoped role names should be confirmed from the current instance configuration before being quoted in other documents.

References

https://c4model.com/ | https://developer.servicenow.com/ (ServiceNow Developer documentation)